All the examples of handler methods you have seen so far have a return type of `void`. The other supported return types are `Task` and any type that implements the `IActionResult` interface, known as action results, which have the role of generating the response. A variety of action results are available for different types of response. For example, instead of rendering a Razor page, you might want to return a file. Or you might want to return an empty response with a specific HTTP status code. Or you might want to redirect the user to a different location.

At this point, you might be wondering how your pages have generated responses when your handler methods haven't returned `IActionResult` types. This is because handler methods that have a return type of `void` or `Task` implicitly return a `PageResult`, an action result that renders the associated Razor page. The following listing shows the `OnPostSearch` handler updated to include an explicit return type of `PageResult`.

Listing 3.33 A handler method explicitly returning an action result

```
public PageResult OnPostSearch(string searchTerm)    ❶
{
    Message = $"You searched for {searchTerm}";
    return new PageResult();                         ❷
}
```

❶ The OnPostSearch method has been altered to return PageResult.

❷ A new PageResult instance is created and returned.

The `PageModel` class includes a number of helper methods that provide a shorthand way of creating action results to save you from using the `new` operator. The next listing shows the use of the `Page()` helper method, which is a wrapper around the expression `new PageResult()`.

Listing 3.34 `Page()` method shorthand for new `PageResult()`

```
public PageResult OnPostSearch(string searchTerm)    ❶
{
    Message = $"You searched for {searchTerm}";
    return Page();                                   ❷
}
```